

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Yuke Brilliant Hestiavin - 5024241016

2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler adalah komponen utama sistem tertanam. Ia membaca kondisi dunia fisik lewat sensor atau tombol, memprosesnya, lalu menggerakkan keluaran seperti LED atau motor. Perangkat ini dipakai di robot, otomasi industri, peralatan rumah tangga, sampai perangkat *Internet of Things* (IoT). Sebelum merancang sistem yang rumit, praktikan perlu menguasai tiga hal dasar: menyiapkan lingkungan pengembangan, mengakses pin masukan dan keluaran, dan membaca besaran analog.

Masalah pertama yang dihadapi adalah pemilihan perangkat dan *framework*. STM32 Blackpill dan ESP32 DevKit punya karakteristik berbeda, begitu pula framework Arduino dan ESP-IDF yang berada pada tingkat abstraksi berbeda. Memahami perbedaan itu menentukan seberapa cepat sebuah prototipe jadi dan seberapa jauh kendali yang bisa diambil atas perangkat keras.

Masalah kedua ada di level rangkaian. Pin masukan yang dibiarkan mengambang (*floating*) terbaca tidak menentu karena derau, dan tombol mekanik menghasilkan beberapa transisi sinyal meski hanya ditekan sekali (*bouncing*). Keduanya bisa membuat program menghitung kejadian yang tidak pernah terjadi. Resistor *pull-up*/*pull-down* dan teknik *debouncing* dipakai untuk menjaga pembacaan tombol tetap andal. Di luar sinyal digital, banyak besaran fisik seperti cahaya, suhu, dan jarak berubah secara kontinu, sehingga dibutuhkan *Analog-to-Digital Converter* (ADC) untuk mengolahnya.

Praktikum Modul 1 ini mencakup seluruh hal tersebut: menyiapkan VSCode dan PlatformIO, menjalankan program Blink pada STM32 dan ESP32, membandingkan pembacaan tombol dengan *pull* eksternal dan internal, menerapkan *debouncing*, dan membaca sensor LDR lewat ADC [1]. Konsep ini dipakai lagi pada modul berikutnya, misalnya untuk antarmuka pengguna, pemantauan cahaya, dan sistem kendali berbasis sensor.

1.2 Dasar Teori

1.2.1 Mikrokontroler dan Sistem Tertanam

Mikrokontroler menggabungkan CPU, memori, dan periferal masukan-keluaran dalam satu chip. Memori Flash menyimpan program, sedangkan RAM menyimpan data sementara selama program berjalan. Periferal seperti GPIO, timer, UART, dan ADC memungkinkan CPU berinteraksi dengan perangkat di luar chip. Sistem tertanam adalah sistem yang memakai komponen tersebut untuk satu fungsi khusus, misalnya membaca sensor lalu menyalakan indikator.

Chip dan board perlu dibedakan. Board pengembangan menambahkan regulator daya, konektor, LED, dan kadang chip USB-to-Serial di sekeliling chip mikrokontroler. Kemampuan diprogram lewat USB, misalnya, berasal dari board dan bukan dari chip mikrokontrolernya.

1.2.2 VSCode, PlatformIO, dan Framework

Visual Studio Code (VSCode) adalah editor kode yang dapat diperluas dengan ekstensi. PlatformIO adalah ekstensi yang menyediakan proyek, *toolchain*, kompilasi, pengunggahan firmware, dan Serial Monitor dalam satu antarmuka untuk banyak board [2]. *Framework* adalah kerangka pustaka dan struktur program untuk mengakses fitur mikrokontroler. Satu chip bisa diprogram dengan framework berbeda hanya dengan mengubah berkas konfigurasi `platformio.ini`.

Struktur proyek PlatformIO terdiri dari `platformio.ini` (board, framework, protokol *upload*), `src/` (kode utama), `include/` (berkas *header*), dan `lib/` (pustaka tambahan).

1.2.3 STM32 Blackpill, Arduino, dan SWD

Blackpill memakai mikrokontroler STM32F401CCU6 dengan CPU ARM Cortex-M4 32-bit dengan FPU, frekuensi hingga 84 MHz, Flash 256 KB, dan RAM 64 KB [3]. LED bawaan terhubung ke pin PC13 dan bersifat aktif LOW: keluaran LOW menyalakan LED, HIGH mematikannya. Tombol pengguna ada di pin PA0.

Framework Arduino untuk STM32 dikenal sebagai STM32duino [4]. Ia menyediakan fungsi `pinMode()`, `digitalWrite()`, dan `delay()` yang sama dengan board Arduino pada umumnya, serta mendukung penamaan pin sesuai label port seperti `PC13`. Di balik layar, framework ini tetap dibangun di atas *Hardware Abstraction Layer* (HAL) milik STMicroelectronics. Board ini tidak punya programmer bawaan, sehingga firmware diunggah lewat ST-Link eksternal melalui protokol *Serial Wire Debug* (SWD) dengan jalur SWDIO, SWCLK, GND, dan 3,3 V.

1.2.4 ESP32, ESP-IDF, dan Komunikasi Serial

ESP32 adalah mikrokontroler 32-bit dual-core (Xtensa LX6) dengan Wi-Fi dan Bluetooth terintegrasi [5]. Board ESP32 DevKit v1 membawa chip USB-to-UART (CP2102 atau CH340) yang menjembatani USB komputer dengan UART ESP32, sehingga firmware dapat diunggah hanya dengan kabel USB.

ESP-IDF (*Espressif IoT Development Framework*) adalah framework resmi Espressif yang dibangun di atas FreeRTOS [6]. Program dimulai dari fungsi `app_main()` yang berjalan sebagai *task* FreeRTOS. Penundaan memakai `vTaskDelay(pdMS_TO_TICKS(ms))`, yang menyerahkan CPU ke *task* lain selama menunggu. Pada framework Arduino untuk ESP32 [7], program disederhanakan menjadi `setup()` dan `loop()`.

1.2.5 GPIO, Pull-up, dan Pull-down

General-Purpose Input/Output (GPIO) dapat dikonfigurasi sebagai keluaran atau masukan digital. Pin masukan yang tidak terhubung ke level tegangan apa pun berada pada kondisi *floating* dan terbaca acak. Resistor menarik pin ke level tertentu saat tombol tidak ditekan:

- **Pull-up**: pin ditarik ke 3,3 V lewat resistor, tombol menghubungkan pin ke GND. Tidak ditekan terbaca HIGH, ditekan terbaca LOW.
- **Pull-down**: pin ditarik ke GND lewat resistor, tombol menghubungkan pin ke 3,3 V. Tidak ditekan terbaca LOW, ditekan terbaca HIGH.

Resistor dapat dipasang eksternal (misalnya 10 kΩ) atau diaktifkan internal lewat `INPUT_PULLUP` dan `INPUT_PULLDOWN`. Saat tombol ditekan pada pull-up 10 kΩ, arus yang mengalir adalah

$$I = \frac{V}{R} = \frac{3{,}3\ \text{V}}{10\,000\ \Omega} = 0{,}33\ \text{mA}. \tag{1}$$

Pin GPIO34 sampai GPIO39 pada ESP32 hanya dapat berfungsi sebagai masukan dan tidak memiliki resistor *pull* internal, sehingga tombol pada pin tersebut wajib memakai resistor eksternal [5].

1.2.6 Bouncing dan Debouncing

Kontak logam pada tombol mekanik memantul saat bersentuhan atau terlepas, sehingga satu tekanan menghasilkan beberapa transisi HIGH ke LOW dalam waktu singkat (orde milidetik). Program yang menghitung setiap tepi sinyal akan mencatat lebih dari satu kejadian. *Debouncing* perangkat lunak mencatat waktu setiap perubahan pembacaan mentah, dan baru menerima keadaan baru jika pembacaan tidak berubah selama jeda tertentu, misalnya 20 sampai 50 ms. Jeda terlalu pendek meloloskan pantulan; jeda terlalu panjang membuat tombol terasa lambat dan bisa melewatkan tekanan cepat.

1.2.7 ADC dan Pembagi Tegangan LDR

ADC mengubah tegangan analog menjadi kode digital melalui pencuplikan, kuantisasi, dan pengodean. ADC N bit memiliki 2^N kode, dari 0 sampai $2^N - 1$. ESP32 memiliki ADC 12 bit (kode 0 sampai 4095) yang dibaca dengan `analogRead()`. Konversi sederhana kode ke tegangan pada modul adalah

$$V_{\text{perkiraan}} = \frac{D}{4095} \times 3{,}3 \text{ V}, \qquad D \in \{0, 1, \ldots, 4095\}. \tag{2}$$

Persamaan ini hanya model sederhana: ketepatan ADC ESP32 dipengaruhi atenuasi, kalibrasi, dan derau, dan responsnya tidak sepenuhnya linear [7]. ADC1 (misalnya GPIO32 sampai 39) lebih disarankan daripada ADC2 karena ADC2 bentrok dengan Wi-Fi.

LDR (*Light Dependent Resistor*) adalah resistor yang hambatannya turun ketika cahaya makin terang. Dipasang antara 3,3 V dan titik ukur, dengan resistor tetap $R = 10$ kΩ dari titik ukur ke GND, tegangan keluarannya mengikuti

$$V_o = V_{\text{in}} \frac{R}{R_{\text{LDR}} + R}. \tag{3}$$

Cahaya yang makin terang menurunkan R_{LDR}, sehingga V_o dan kode ADC naik. Nilai ini menunjukkan perubahan relatif pencahayaan; untuk satuan lux dibutuhkan kalibrasi sensor.

2 Tugas Pendahuluan

1. **Perbedaan mendasar framework Arduino dan ESP-IDF, beserta kelebihan dan kekurangannya.**

   Arduino menyederhanakan pemrograman dengan API tingkat tinggi, sedangkan ESP-IDF memberi kendali rinci atas fitur ESP32 dan sistem operasinya (FreeRTOS).

Aspek	Arduino	ESP-IDF
Titik masuk program	`setup()` dan `loop()`	`app_main()`, berjalan sebagai *task* FreeRTOS
Penundaan	`delay(ms)`	`vTaskDelay(pdMS_TO_TICKS(ms))`
Tingkat abstraksi	Tinggi, API sederhana seperti `digitalWrite()`	Rendah sampai menengah, konfigurasi perangkat dan sistem lebih rinci
Akses RTOS	Tersembunyi atau terbatas	Langsung (*task*, *queue*, *semaphore*)
Kelebihan	Mudah dipelajari, prototipe cepat, pustaka dan contoh melimpah	Kendali penuh atas multitasking, memori, daya, dan konfigurasi sistem
Kekurangan	Abstraksi membatasi optimasi dan akses fitur tingkat rendah	Kurva belajar curam, kode dan konfigurasi lebih panjang
Cocok untuk	Praktikum, prototipe, proyek sederhana	Produk IoT dan sistem dengan kebutuhan teknis khusus

Tabel 1: Perbandingan framework Arduino dan ESP-IDF

Arduino untuk ESP32 dibangun di atas ESP-IDF, sehingga fitur ESP-IDF dan FreeRTOS tetap dapat dipanggil dari proyek Arduino.

2. **Mengapa STM32 Blackpill memerlukan ST-Link eksternal, sedangkan ESP32 dapat diprogram lewat USB.**

   Perbedaannya ada pada rangkaian board, bukan pada chip mikrokontrolernya.

   - **STM32 Blackpill** tidak memiliki chip USB-to-Serial maupun programmer bawaan. Pin SWD (SWDIO dan SWCLK) diekspos langsung dari chip untuk pemrograman dan *debugging*, sehingga dibutuhkan ST-Link eksternal sebagai jembatan antara USB komputer dan SWD. Pengurangan komponen ini membuat board murah dan kecil.
   - **ESP32 DevKit** memiliki chip USB-to-UART (CP2102 atau CH340) di board. Chip ini mengubah data USB dari komputer menjadi sinyal UART yang dimengerti *bootloader* internal ESP32. Rangkaian transistor otomatis pada board juga mengatur pin `EN` dan `GPIO0` dari sinyal DTR/RTS sehingga ESP32 masuk mode unggah tanpa menekan tombol.
   - **Catatan:** chip STM32F401 juga punya *bootloader* bawaan (misalnya DFU lewat USB dengan mengatur BOOT0 lalu *reset*). Jadi ST-Link bukan satu-satunya cara memprogram Blackpill, tetapi ia unggul karena sekaligus menyediakan *debugging*.

3. **Kondisi HIGH/LOW pada pull-up dan pull-down saat tombol tidak ditekan dan ditekan.**

Konfigurasi	Tidak ditekan	Ditekan
Pull-up (tombol ke GND)	HIGH	LOW
Pull-down (tombol ke 3,3 V)	LOW	HIGH

Tabel 2: Level logika pada pin GPIO

Pada *pull-up*, resistor menarik pin ke 3,3 V saat tombol terbuka sehingga terbaca HIGH. Saat ditekan, tombol menyambungkan pin ke GND dengan hambatan jauh lebih kecil dari resistor, sehingga pin terbaca LOW (logika aktif LOW). Pada *pull-down* berlaku sebaliknya: resistor menarik pin ke GND (LOW), dan penekanan menyambungkannya ke 3,3 V (HIGH, aktif HIGH). Dalam kedua konfigurasi, resistor mencegah kondisi *floating* dan membatasi arus ketika tombol ditutup. Hal yang sama berlaku untuk resistor internal, kecuali pada GPIO34 sampai 39 ESP32 yang tidak memilikinya.

4. **Fenomena bouncing, penyebabnya, dan minimal dua cara mengatasinya.**

   Bouncing terjadi karena kontak logam di dalam tombol bersifat elastis. Saat ditekan atau dilepas, kontak memantul beberapa kali sebelum diam, sehingga sinyal berubah HIGH ke LOW berulang selama beberapa milidetik. Mikrokontroler yang cepat membaca setiap pantulan itu sebagai penekanan tersendiri.

   Cara mengatasinya:

   (a) **Debouncing perangkat lunak:** terima perubahan status hanya jika pembacaan stabil selama waktu tertentu, misalnya 10 sampai 50 ms, dengan menyimpan waktu perubahan terakhir memakai `millis()`.
   (b) **Debouncing perangkat keras:** pasang filter RC pada pin tombol untuk meredam pulsa singkat, lalu Schmitt trigger untuk mengembalikan transisi logika yang tegas.

5. **Cara kerja ADC dan pengaruh resolusi bit terhadap presisi.**

   ADC mengubah tegangan analog menjadi angka digital lewat tiga tahap:

   (a) **Sampling**: mengambil nilai tegangan pada saat tertentu.
   (b) **Kuantisasi**: memetakan nilai tersebut ke tingkat diskret terdekat.
   (c) **Pengodean**: menyatakan tingkat itu sebagai bilangan biner.

   ADC n bit memiliki 2^n tingkat dengan kode 0 sampai $2^n - 1$. Untuk rentang masukan 0 sampai V_{ref}, ukuran satu langkah (resolusi) ideal adalah

$$\Delta V = \frac{V_{\text{ref}}}{2^n}. \tag{4}$$

   Dengan $V_{\text{ref}} = 3{,}3$ V:

Resolusi	Jumlah tingkat	ΔV
8 bit	256	12,89 mV
10 bit	1024	3,22 mV
12 bit	4096	0,806 mV

Tabel 3: Resolusi ADC untuk $V_{\text{ref}} = 3{,}3$ V

   Makin tinggi resolusi, makin kecil perubahan tegangan yang masih dapat dibedakan. Meski begitu, akurasi aktual tetap dibatasi derau, kualitas tegangan referensi, dan kalibrasi, jadi resolusi tinggi tidak otomatis berarti hasil lebih akurat.

6. **Set-up VSCode, PlatformIO (ESP-IDF dan STM32), dan Serial Plotter.**

   Langkah yang dilakukan:

   (a) Memasang VSCode, lalu ekstensi *PlatformIO IDE* lewat menu *Extensions*. Setelah PlatformIO Core selesai terpasang, PlatformIO Home dapat dibuka (Gambar 1).
   (b) Membuat satu proyek Modul 1. Untuk STM32 Blackpill, `platformio.ini` diisi seperti di bawah, dengan kode Blink pada `src/main.cpp`. Pengunggahan membutuhkan ST-Link dan kabel SWD yang benar.

```
1 [env:blackpill_f401cc]
2 platform = ststm32
```

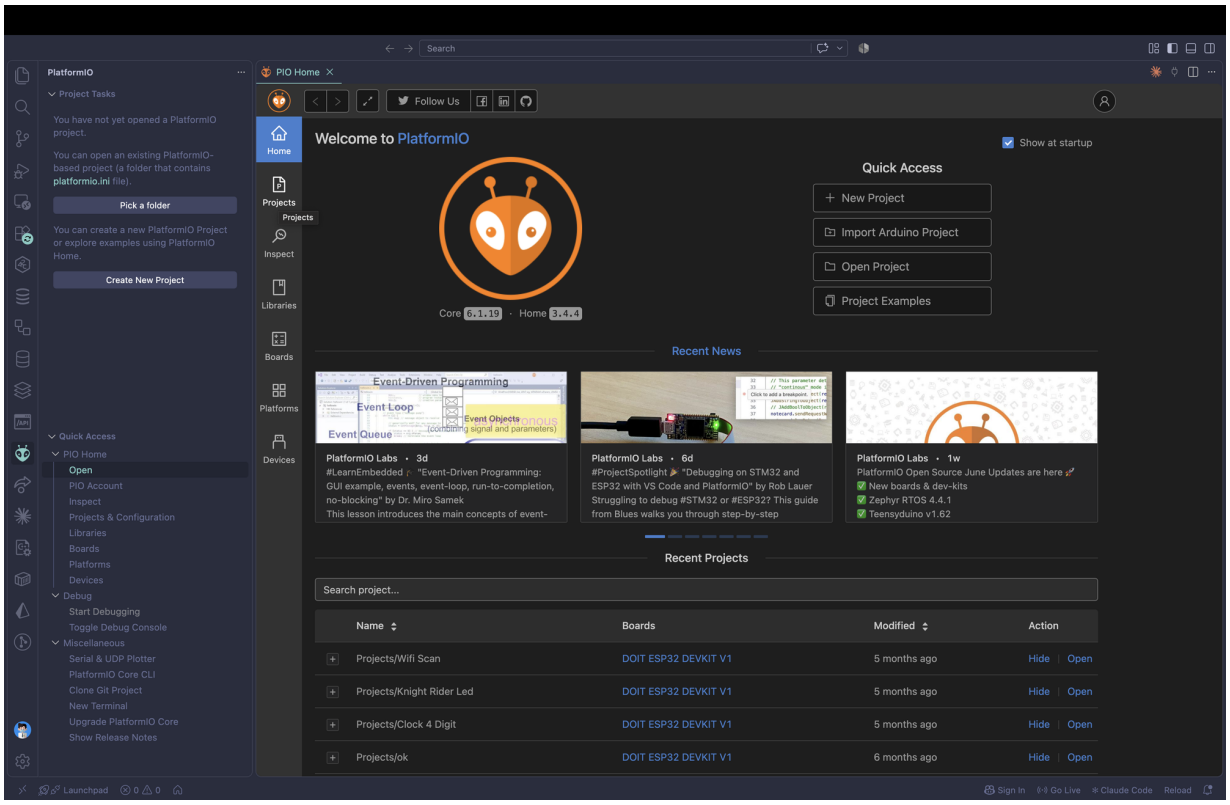
```
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

(c) Untuk ESP32 dengan ESP-IDF, `platformio.ini` pada proyek yang sama diganti dan kode Blink ESP-IDF ditulis pada `src/main.c` (berkas `main.cpp` lama dihapus). PlatformIO mengunduh paket dan memperbarui konfigurasi sehingga perlu ditunggu sampai selesai sebelum *Build*. Untuk *Upload*, ESP32 disambungkan dengan kabel USB data dan driver CP210x/CH340 sudah terpasang.

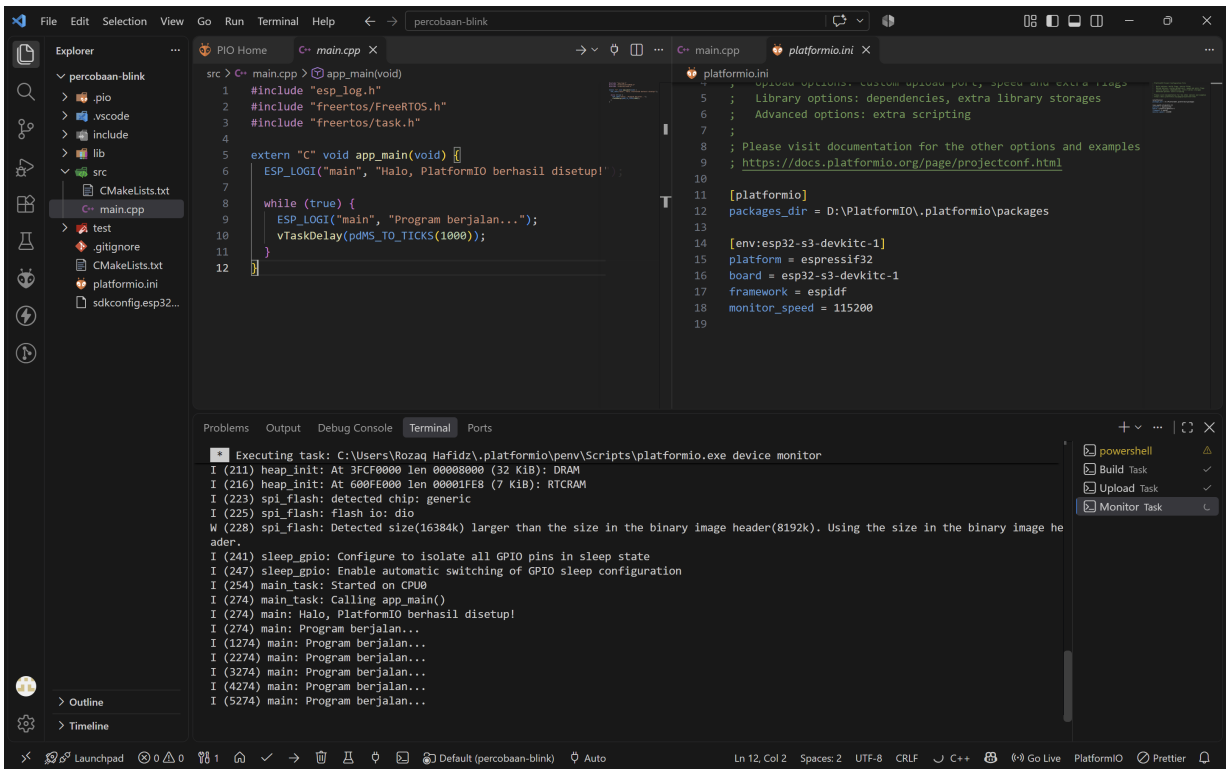
```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

(d) Memasang ekstensi *Serial Plotter*, memilih port board, dan menyamakan *baud rate* dengan firmware (115200 bps).

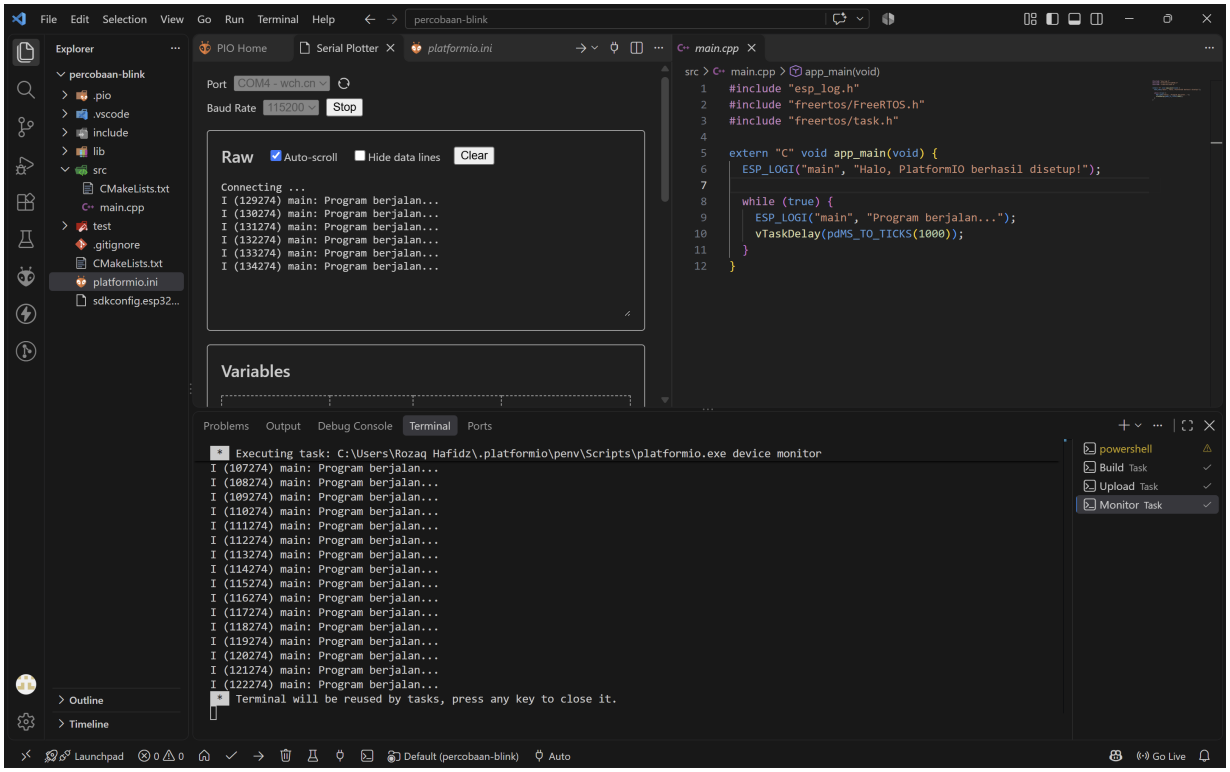
Hasil set-up ditunjukkan pada Gambar 1 sampai Gambar 3.



Gambar 1: VSCode dengan ekstensi PlatformIO terpasang



Gambar 2: Build, Upload, dan Monitor proyek ESP32 dengan ESP-IDF (set-up dikerjakan bersama dalam kelompok)



Gambar 3: Ekstensi Serial Plotter pada VSCode (set-up dikerjakan bersama dalam kelompok)

Daftar Pustaka

[1] Laboratorium Robotika dan Sistem Cerdas, *Modul 1: Pengantar Mikrokontroler, Praktikum Sistem Mikroprosesor dan Sistem Tertanam*, https://github.com/b401-robotics/praktikum-sismik-embed/blob/main/Modul_1/README.md, 2026.

[2] PlatformIO, *PlatformIO Documentation*, https://docs.platformio.org/, Diakses: 2026.

[3] STMicroelectronics, *RM0368: STM32F401xB/C and STM32F401xD/E advanced Arm-based 32-bit MCUs reference manual*, 2015.

[4] STM32duino, *Arduino Core for STM32*, https://github.com/stm32duino/Arduino_Core_STM32, Diakses: 2026.

[5] Espressif Systems, *ESP32 Series Datasheet*, https://www.espressif.com/en/support/documents/technical-documents, Diakses: 2026.

[6] Espressif Systems, *ESP-IDF Programming Guide*, https://docs.espressif.com/projects/esp-idf/, Diakses: 2026.

[7] Espressif Systems, *Arduino-ESP32 Core Documentation*, https://docs.espressif.com/projects/arduino-esp32/, Diakses: 2026.